

CITS5017 Deep Learning

Topic 5: Representation learning and dimension reduction using Autoencoders

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)

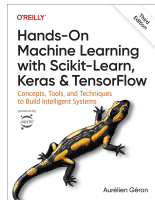
Semester 2, 2026



THE UNIVERSITY OF
WESTERN
AUSTRALIA

Chapters 17.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Autoencoders, GANs, and Diffusion Models

Here are the main topics we will briefly cover:

- What are Autoencoders?
- Efficient data representations
- Encoder and decoder in an autoencoder
- Performing PCA using an *undercomplete* linear autoencoder
- Stacked autoencoders and how to implement one using Keras in TensorFlow
- Tying the weights in a stacked autoencoder and visualizing the reconstruction
- Unsupervised pre-training using stacked autoencoders
- Training one autoencoder at a time

(*Diffusion models* will not be covered in the unit)

What Are Autoencoders?

- Autoencoders are artificial neural networks capable of learning efficient representations of the input data, called *latent representations codings*, without any supervision (i.e., the training set is unlabelled). These codings typically have a *much lower dimensionality* than the input data, making autoencoders useful for *dimensionality reduction*.
- More importantly, autoencoders act as *feature detectors*, and they can be used for unsupervised pre-training of deep neural networks.
- Lastly, some autoencoders are *generative models*: they are capable of randomly generating new data that looks very similar to the training data. For example, you could train an autoencoder on pictures of faces, and it would then be able to generate new faces. However, the generated images are usually fuzzy and not entirely realistic.

Efficient Data Representations

Which of the following number sequences do you find the easiest to memorize?

- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68
- 50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14

While the first sequence is shorter and seemingly easier to remember, the second sequence has a notable pattern (decreasing even numbers from 50 to 14).

Just like expert chess players being able to detect patterns in chess games, we can train an autoencoder to convert the inputs to efficient latent representations and then spits out something that (hopefully) look very close to the inputs.

Efficient Data Representations (cont.)

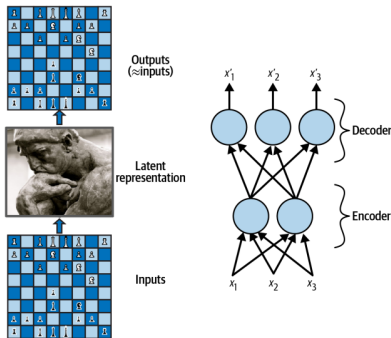


Figure 17-1. The chess memory experiment (left) and a simple autoencoder (right)

- An autoencoder typically has the same architecture as a **Multi-Layer Perceptron (MLP)**, except that the number of neurons in the output layer must be equal to the number of neurons in the input layer.
- In the example here, there is just one hidden layer composed of two neurons (the **encoder**), and one output

layer composed of three neurons (the **decoder**). The outputs are often called the **reconstructions** since the autoencoder tries to reconstruct the inputs, and the cost function contains a **reconstruction loss** that penalizes the model when the reconstructions are different from the inputs.

Efficient Data Representations (cont.)

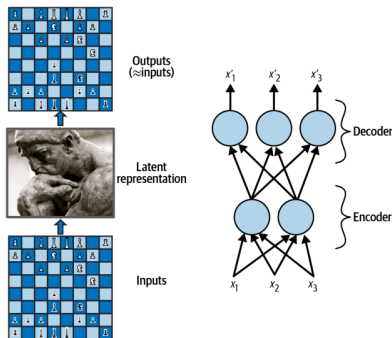


Figure 17-1. The chess memory experiment (left) and a simple autoencoder (right)

- Because the internal representation has a lower dimensionality than the input data (it is 2D instead of 3D), the autoencoder is said to be *undercomplete*.
- An undercomplete autoencoder cannot trivially copy its inputs to the codings. So, it must learn to find a way to output a copy of its inputs. i.e., it is forced to learn the most important features in the input data (and drop the unimportant ones).

Performing PCA with an Undercomplete Linear Autoencoder

If the autoencoder uses only **linear activations** and the cost function is the **Mean Squared Error (MSE)**, then it can be shown that it ends up performing **Principal Component Analysis** (see Chapter 8).

Example: Build a simple linear autoencoder to perform PCA:

```
import tensorflow as tf

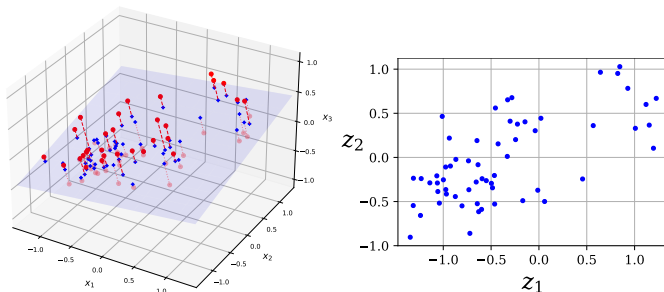
encoder = tf.keras.models.Sequential([tf.keras.layers.Dense(2)])
decoder = tf.keras.models.Sequential([tf.keras.layers.Dense(3)])
autoencoder = tf.keras.Sequential([encoder, decoder])

optimizer = tf.keras.optimizers.SGD(learning_rate=0.5)
autoencoder.compile(loss="mse", optimizer=optimizer)
```

Notes: (i) the autoencoder consists of two subcomponents: the **encoder** and the **decoder**. (ii) The autoencoder's number of outputs is equal to the number of inputs (i.e., 3). (iii) To perform PCA, we do not use any activation function (i.e., all neurons are linear), and the cost function is the MSE.

Performing PCA with an Undercomplete Linear Autoencoder (cont.)

Now let's generate the same 3D data used in Chapter 8:



Left: The 3D training set. Bright/light red dots: 3D points above/below the light blue plane; blue markers: the projections of the 3D points onto the plane. **Right:** The approximate PCA performed by the undercomplete linear autoencoder. (compare the blue markers in the two diagrams)

```
X_train = [...] # shape of X_train is (60,3) in the example above
history = autoencoder.fit(X_train, X_train, epochs=500, verbose=False)
codings = encoder.predict(X_train)
```

Stacked Autoencoders

- Just like other neural networks we have discussed, autoencoders can have multiple hidden layers. In this case they are called *stacked autoencoders* (or *deep autoencoders*).
- Adding more layers helps the autoencoder learn more complex codings. However, one must be careful not to make the autoencoder too powerful. Imagine an encoder so powerful that it just learns to map each input to a single arbitrary number (and the decoder learns the reverse mapping). Obviously such an autoencoder will reconstruct the training data perfectly, but it will not have learned any useful data representation in the process (and it is unlikely to generalize well to new instances).

Stacked Autoencoders (cont.)

- The architecture of a stacked autoencoder is typically symmetrical with regards to the **central hidden layer (the coding layer)**. To put it simply, it looks like a sandwich, e.g., an autoencoder for MNIST may have 784 inputs, then a hidden layer with 100 neurons, a central hidden layer of 30 neurons, then another hidden layer with 100 neurons, and an output layer with 784 neurons (Figure 17-3).

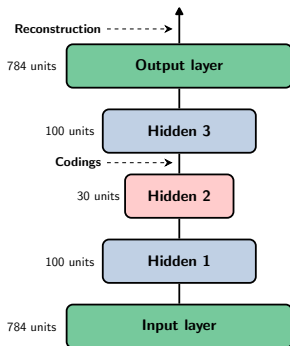


Figure 17-3. Stacked autoencoder

Implementing a Stacked Autoencoder Using Keras

You can implement a stacked autoencoder very much like a regular deep MLP. The code below is for the Fashion MNIST dataset.

```
stacked_encoder = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[28,28]),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
stacked_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
stacked_ae = tf.keras.Sequential([stacked_encoder, stacked_decoder])

stacked_ae.compile(loss="mse", optimizer="nadam")
history = stacked_ae.fit(X_train, X_train, epochs=20,
                        validation_data=(X_valid, X_valid))
```

This model (`stacked_ae`) has 163,814 trainable parameters.

Implementing a Stacked Autoencoder Using Keras (cont.)

Visualizing the Reconstructions

One way to ensure that an autoencoder is properly trained is to compare the inputs and the outputs: the differences should not be too significant.



Figure 17-4. Original images (top) and their reconstructions (bottom)

The reconstructions are recognizable, but a bit too lossy. We may need to train the model for longer, or make the encoder and decoder deeper, or make the codings larger.

Implementing a Stacked Autoencoder Using Keras (cont.)

Visualizing Using T-SNE

Once we have reduced the dataset's dimensionality, we can use another dimensionality reduction algorithm to see how well the lower-dimensional data is for the downstream classification task. T-SNE¹ is a suitable algorithm as it reduces the data down to 2D for visualization.

```
from sklearn.manifold import TSNE
X_valid_compressed = stacked_encoder.predict(X_valid)
tsne = TSNE()
X_valid_2D = tsne.fit_transform(X_valid_compressed)
```

¹ *t-distributed stochastic neighbour embedding* (proposed by van der Maaten and Hinton in 2008) reduces dimensionality while trying to keep similar instances close and dissimilar instances apart.

Implementing a Stacked Autoencoder Using Keras (cont.)

Visualizing Using T-SNE (cont.)



Figure 17-5. Fashion MNIST visualization using an autoencoder followed by t-SNE

Unsupervised Pretraining Using Stacked Autoencoders

So, autoencoders can be used for dimensionality reduction. Another application is for unsupervised pretraining.

As discussed in Chapter 11, if you are tackling a complex supervised task but you do not have a lot of labelled training data, one solution is to find a neural network that performs a similar task and reuse its lower layers. This makes it possible to train a high-performance model using little training data because your neural network won't have to learn all the low-level features; it will just reuse the feature detectors learned by the existing network.

Similarly, we can firstly train a stacked autoencoder using all the data. This training process does not require the data to be labelled. Once the stacked autoencoder is trained, we can reuse its lower layers in our neural network for the actual task. As our neural network has fewer layers that require training, it can cope with fewer labelled training data.

Unsupervised Pretraining Using Stacked Autoencoders (cont.)

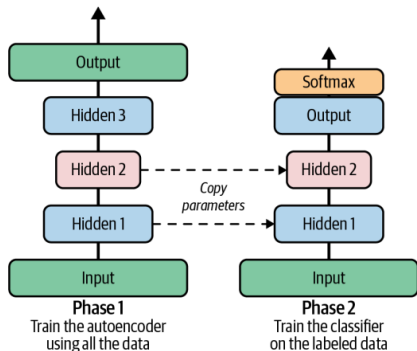


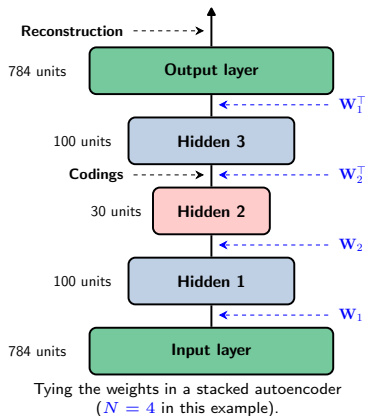
Figure 17-6. Unsupervised pretraining using autoencoders.

There is nothing special about the implementation: just train an autoencoder using all the training data, then reuse its encoder layers to create a new neural network for your actual task and train it using the labelled data.

Stacked Autoencoders: Tying the Weights

When an autoencoder is neatly symmetrical, like the one we just built, a common technique is to *tie the weights* of the decoder layers to the weights of the encoder layers. This *halves the number of weights* in the model, speeding up training and limiting the risk of overfitting.

Stacked Autoencoders: Tying the Weights (cont.)



Specifically, if the autoencoder has a total of N layers (not counting the input layer), and $\mathbf{W}_\ell$ represents the connection weights of the ℓ^{th} layer (e.g., layer 1 is the first hidden layer, layer $\frac{N}{2}$ is the coding layer, and layer N is the output layer), then the decoder layer weights can be defined simply as:

$$\mathbf{W}_{N-\ell+1} = \mathbf{W}_\ell^\top$$

with $\ell = 1, 2, \dots, N/2$.

In this example, only $\mathbf{W}_1$, $\mathbf{W}_2$, and the bias terms need to be learned.

Stacked Autoencoders: Tying the Weights (cont.)

(Note the modification (line # ***) to Géron's code)

```
class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.biases = self.add_weight(
            name="bias",
            shape=[self.dense.input.shape[-1]],      # ***
            initializer="zeros")
        super().build(batch_input_shape)

    def call(self, inputs):
        Z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)
        return self.activation(Z + self.biases)

dense_1 = tf.keras.layers.Dense(100, activation="relu")
dense_2 = tf.keras.layers.Dense(30, activation="relu")
```

Stacked Autoencoders: Tying the Weights (cont.)

```
tied_encoder = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[28,28]),
    tf.keras.layers.Flatten(),
    dense_1,
    dense_2
])
tied_decoder = tf.keras.Sequential([
    DenseTranspose(dense_2, activation="relu"),
    DenseTranspose(dense_1),
    tf.keras.layers.Reshape([28, 28])
])

tied_ae = tf.keras.Sequential([tied_encoder, tied_decoder])
tied_ae.compile(loss="mse", optimizer="nadam")
history = tied_ae.fit(X_train, X_train, epochs=10,
                     validation_data=(X_valid, X_valid))
```

Compared to **stacked_ae** which has 163,814 trainable parameters, **tied_ae** only has 82,414 trainable parameters (number of parameters that require training is almost halved). It also achieves a slightly lower reconstruction error than **stacked_ae**.

Stacked Autoencoders: Tying the Weights (cont.)

Reconstruction from the tied_ae model



Original images (top) and their reconstructions (bottom)

Training One Autoencoder at a Time

It is possible to train one shallow autoencoder at a time, then stack all of them into a single stacked autoencoder (hence the name). For example,

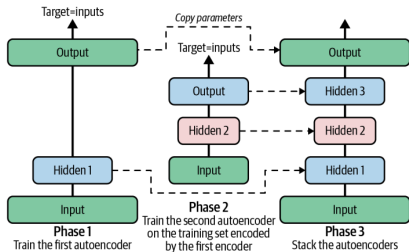


Figure 17-7. Training one autoencoder at a time

- **Phase 1:** Train the first autoencoder to learn a compressed representation $\mathbf{X}_{(1)}$ of the inputs.
- **Phase 2:** Using $\mathbf{X}_{(1)}$ as input data, train the second autoencoder to learn an even more compressed representation $\mathbf{X}_{(2)}$.
- **Phase 3:** Finally, we build a big sandwich using all these autoencoders.

There can be more phases if the autoencoder is very deep.

Training One Autoencoder at a Time (cont.)

```
def train_autoencoder(n_neurons, X_train, X_valid, n_epochs=10,
                      output_activation=None):
    n_inputs = X_train.shape[-1]
    encoder = tf.keras.layers.Dense(n_neurons, activation="relu")
    decoder = tf.keras.layers.Dense(n_inputs, activation=output_activation)
    autoencoder = tf.keras.Sequential([encoder, decoder])
    autoencoder.compile(loss="mse", optimizer="nadam")
    autoencoder.fit(X_train, X_train, epochs=n_epochs,
                    validation_data=(X_valid, X_valid))
    return encoder, decoder, encoder(X_train), encoder(X_valid)

X_train_flat = tf.keras.layers.Flatten()(X_train)
X_valid_flat = tf.keras.layers.Flatten()(X_valid)

# Train the first autoencoder using X_train_flat and X_valid_flat.
# X_train_enc1 and X_valid_enc1 are the compressed representations.
enc1, dec1, X_train_enc1, X_valid_enc1 = train_autoencoder(
    100, X_train_flat, X_valid_flat)

# Train the second autoencoder using X_train_enc1 and X_valid_enc1.
enc2, dec2, _, _ = train_autoencoder(
    30, X_train_enc1, X_valid_enc1, output_activation="relu")

# Stack up to form the final autoencoder.
stacked_ae_1_by_1 = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    enc1, enc2, dec2, dec1,
    tf.keras.layers.Reshape([28, 28])
])
```

Training One Autoencoder at a Time (cont.)

```
# If needed, we can then continue training the full stacked autoencoder
# for a few epochs
stacked_ae_1_by_1.compile(loss="mse", optimizer="nadam")
history = stacked_ae_1_by_1.fit(X_train, X_train, epochs=5,
                                validation_data=(X_valid, X_valid))

# visualize the reconstruction
plot_reconstructions(stacked_ae_1_by_1) # see the author's Jupyter
                                         # notebook file
plt.show()
```



Original images (top) and their reconstructions (bottom)

Summary

- Understand that autoencoders can be trained and used as a way to efficiently represent data (analogous to PCA).
- Understand the two main parts in an autoencoder: the *encoder* and the *decoder*.
- Know how to implement an *undercomplete* linear autoencoder in TensorFlow to perform PCA on the input data.
- Understand stacked autoencoders and know how to implement one using TensorFlow.
- Understand the concept of tying the weights in a stacked autoencoder and know how to visualize the reconstruction.
- Understand how to use stacked autoencoders in unsupervised pre-training tasks.

For the next two topics...

Read up to Chapter 17 on **Autoencoders, GANs, and Diffusion Models** (from page 647 to page 667; excluding the *Sparse Autoencoders* section)

And that's all for this lecture.

Have a good week.